

REQUIREMENTS-DRIVEN SEMANTIC GATEWAY ARCHITECTURE

Integrating JTBD · Persona Needs · Functional & Non-Functional Requirements
into the Ontology, SHACL/SWRL Rules Store & Domain Vector Databases

STPA/STAMP Hardened Edition | Extension to Semantic Gateway Architecture v1

DRAFT v2.0

Executive Summary

The Requirements-Driven Semantic Gateway Architecture (RDSG) is a system-theoretic framework for governing AI behavior at enterprise scale. It does not attempt to make AI models "reliable" in the stochastic sense — an objective that the STAMP theoretical framework, as developed by Professor Nancy Leveson at MIT, correctly identifies as insufficient for complex, software-intensive systems. Instead, the RDSG makes AI-embedded systems controllable. It constructs a formal, hierarchical control structure around AI agents, enforces safety constraints through machine-readable semantics, and closes the loop from stakeholder intent to validated AI output with complete traceability.

This edition integrates the STPA (System-Theoretic Process Analysis) methodology directly into the RDSG architecture. STPA is the leading hazard analysis technique derived from STAMP. It identifies Unsafe Control Actions (UCAs) across all four failure modes — not just wrong outputs, but missing outputs, mis-timed outputs, and outputs applied too long or stopped too soon — and generates causal scenarios that drive architectural hardening decisions. The result is an AI governance architecture that can scale with emergent complexity, govern agentic behaviors across distributed control hierarchies, and remain analytically defensible as capabilities expand.

Why Controllability — Not Reliability — Is the Right Goal

Traditional AI safety discourse focuses on reliability: measuring how often an AI produces correct outputs. This framing fails at scale. A system can be statistically reliable and still produce catastrophic outputs in edge cases that were never covered by training data. Prof. Leveson's foundational insight — that accidents in complex systems arise from unsafe interactions among components that have not individually failed — applies with full force to AI systems.

The RDSG shifts the goal: instead of measuring AI reliability, the architecture enforces behavioral constraints. The AI operates inside a control envelope defined by the Requirements Ontology, governed by SHACL shapes and SWRL rules, validated by AcceptanceCriteria before output delivery. Emergent behaviors are constrained not by predicting every possible output, but by making the control structure robust enough to catch and contain any output that violates a stakeholder-defined safety constraint.

As AI capabilities expand — from single-turn completions to multi-step agentic chains — the RDSG control structure scales with them. Each new agent is a new controlled process that inherits the existing constraint hierarchy. Each new job-to-be-done is a new requirement that seeds new SHACL shapes and SWRL rules. Complexity does not erode safety; it is absorbed by the control structure.

Key architectural capabilities of the STPA/STAMP-hardened RDSG include:

- **Full STAMP Control Structure alignment:** the Requirements Ontology and JTBD framework act as the top-level controller, issuing behavioral control actions downward through SHACL/SWRL enforcement to AI agents and domain vector databases.
- **Complete UCA coverage across all four STPA failure modes:** missing control action, hazardous control action, mis-timed control action, and control action applied too long or stopped too soon — all now systematically mapped to SWRL rule patterns and gateway enforcement logic.
- **Simplex Action Gateway at Zone 6:** a formal irreversibility checkpoint that halts AI-triggered external writes and API calls, requiring human-in-the-loop handshake approval before any action that cannot be undone is executed.

- Temporal SWRL Rules (TMP series): a new class of rules governing sequence ordering, timeout enforcement, and safe-state fallback — extending requirement-driven behavior from static content validation into time-critical operational domains.
- Human-System Process Model Synchronization: a formal mechanism for detecting and flagging stakeholder misalignment in the elicitation pipeline, treating human process model divergence as a first-class system hazard — not an AI failure.
- STPA Elicitation Integration: UCA definition is embedded in Stage 1 of the Requirements Capture Pipeline. Every Functional Job now requires stakeholders to define what constitutes an unsafe absence, unsafe presence, unsafe timing, and unsafe duration of the AI's response.

1. The Problem: Requirements as First-Class Semantic Citizens

Most enterprise AI deployments treat requirements as upstream artifacts that live in product management tools and are forgotten the moment a system is built. The result: the AI operates without knowing why it exists, who it serves, what job it is trying to accomplish, or what constitutes success for that job. When the AI fails, there is no requirements baseline to trace the failure back to.

This extension to the Semantic Gateway Architecture solves that by treating requirements as ontology-defined entities with formal structure, machine-readable semantics, and direct linkage to the SHACL constraints, SWRL rules, and vector database indices that govern AI behavior. Requirements do not sit in a separate system — they become the semantic scaffolding through which every other architectural element is organized and validated.

Governing Principle

A requirement is not a document. It is an ontology individual of class Requirement, with typed properties for persona, job, outcome, acceptance criterion, domain, and constraint linkage. It lives in the triplestore, is enforced by SHACL shapes, activates SWRL rules, and seeds the domain vector databases. Every AI action is traceable to one or more governing requirements.

Under the STAMP model, every requirement is also a constraint in the hierarchical control structure. The Requirements Ontology is the top-level controller. SHACL shapes and SWRL rules are its control actions. AI agents and vector databases are the controlled processes. Feedback flows upward through AcceptanceCriteria evaluation and RequirementsFindings. The architecture does not rely on AI reliability — it relies on control structure integrity.

1A. RDSG as a STAMP Hierarchical Control Structure

STAMP (System-Theoretic Accident Model and Processes) models accidents not as failure chains but as the result of inadequate enforcement of safety constraints within a hierarchical control structure. Every level of the structure issues control actions downward and receives feedback upward. Accidents occur when the control structure breaks down: process models become inconsistent with reality, control actions are missing or mis-timed, feedback loops are incomplete, or safety constraints are not enforced.

The RDSG is architecturally isomorphic to a STAMP control structure. The mapping below makes this explicit, grounding every architectural zone in a formally defined STAMP role:

STAMP Element	RDSG Implementation	Architectural Zone
Top-Level Controller	Requirements Ontology + JTBD Framework — issues behavioral mandates as SHACL shapes and SWRL rules	Zone 3
Control Actions (downward)	SHACL shapes evaluate SPOs; SWRL rules fire on requirement class membership; CAD compilation instructions govern agent behavior	Zone 2, 3
Controlled Process (primary)	AI Agents receiving Compiled AI Directives (CADs) and executing domain retrievals and response generation	Zone 5
Controlled Process (secondary)	Domain Vector Databases — retrieval behavior governed by Requirements-Derived Query Envelopes (RDQEs)	Zone 5
Process Model (controller)	Requirement Context Bundle (RCB) — the gateway's synchronized view of active persona, job, scenario, constraints, and acceptance criteria for this interaction	Zone 2
Process Model (human)	Stakeholder's understanding of system capabilities, job structure, and expected outputs — subject to misalignment (see Section 7A)	Zone 1
Feedback (upward)	AcceptanceCriteria evaluation results, RequirementsFindings logs, OntologyGap detections, and RequirementsReview triggers flowing back to domain stewards	Zone 4, Zone 3
Safety Constraints	ControlRequirement individuals with enforcedBy linkage to specific SHACL shapes and SWRL rules — machine-executable, traceable to governing regulation or stakeholder obligation	Zone 3
Actuators (constraint enforcement)	Service Facade — evaluates all agent responses against active AcceptanceCriteria before delivery; recycles non-passing responses with structured correction	Zone 4
External Boundary / Legacy	Simplex Action Gateway at Zone 6 — formal irreversibility checkpoint governing all AI-triggered external writes and API calls	Zone 6

1A.1 Control Structure Diagram: Functional Hierarchy

The following narrative describes the RDSG control hierarchy from top to bottom:

- Level 1 — Requirements Governance (Top Controller): Domain stewards and compliance officers activate, version, and retire Requirement individuals. STPA analysts define UCAs and causal scenarios at this level. Governance gaps and RequirementsReview triggers are resolved here.
- Level 2 — Semantic Gateway (Primary Enforcement Controller): Receives Structured Prompt Objects (SPOs). Executes Stage 0 Requirement Resolution to build the RCB. Fires SHACL evaluation and SWRL rule execution. Compiles Compiled AI Directives (CADs). Acts as the primary behavioral enforcement controller for all AI agents in Zones 4 and 5.
- Level 3 — Service Facade (Secondary Controller): Receives CADs and dispatches to domain agents. Evaluates all responses against AcceptanceCriteria. Routes to Simplex Action Gateway for any external write. Returns structured correction instructions for non-passing responses. Sends acceptance/rejection findings upward as feedback.
- Level 4 — AI Agents + Vector DBs (Controlled Processes): Operate under CAD directives. Retrieve from vector DBs governed by RDQE metadata filters. Generate responses subject to Service Facade validation before delivery to the user.

- Level 5 — Zone 6 Legacy Integration (Actuator Layer): Executes external API calls and write operations. Gated by Simplex Action Gateway. All actions annotated with Requirement IDs for end-to-end traceability. Irreversible actions require human handshake before execution.

2. The Requirements Ontology

The Requirements Ontology is a domain module in the federated ontology stack, positioned above the upper ontology and below all domain-specific ontologies. It provides the shared vocabulary for expressing what people need, why they need it, and what good looks like — in a form the Semantic Gateway can reason over. Under the STAMP model, every class in this ontology corresponds to a constraint type in the hierarchical control structure.

2.1 Core Class Hierarchy

Ontology Class	Superclass	What It Represents
Requirement	owl:Thing	Root class for all requirements — any stated need that must be satisfied by the system
PersonaRequirement	Requirement	A need tied to a specific Persona, derived from their role, responsibilities, and domain context
JobRequirement	PersonaRequirement	A need derived from a specific Job-to-Be-Done that the persona is trying to accomplish
OutcomeRequirement	JobRequirement	The desired end-state — the 'so that' clause of the job
FunctionalRequirement	Requirement	A specific capability the system must provide (what it must do)
NonFunctionalRequirement	Requirement	A quality attribute — performance, reliability, usability — governing how it must do it
ComplianceRequirement	Requirement	A requirement derived from regulatory obligation, standard, or policy
ControlRequirement	ComplianceRequirement	A specific control mechanism — often 1:1 with a SHACL shape or SWRL rule
ProcessRequirement	Requirement	A process step or workflow pattern that must be supported within a domain
ScenarioRequirement	Requirement	Scenario-specific instantiation capturing context, preconditions, and expected behavior
UCAConstraint	ControlRequirement	NEW (STPA): A control requirement derived directly from an identified Unsafe Control Action — specifies the positive constraint the system must satisfy to prevent the UCA from occurring
AcceptanceCriterion	owl:Thing	A testable condition that, when met, proves a requirement is satisfied

2.2 Core Properties

Property	Domain	Range	Meaning
req:forPersona	Requirement	Persona	The persona whose need this requirement expresses
req:describesJob	JobRequirement	Job	The JTBD this requirement is derived from
req:targetsOutcome	OutcomeRequirement	DesiredOutcome	The outcome the persona wants to achieve
req:inDomain	Requirement	BusinessDomain	The business domain this requirement belongs to
req:governedBy	Requirement	ComplianceObligation	The regulation, standard, or policy that creates this requirement
req:enforcedBy	ControlRequirement	SHACLShape SWRLRule	The shape or rule that implements this control
req:hasCriterion	Requirement	AcceptanceCriterion	One or more testable acceptance criteria
req:priority	Requirement	xsd:string	MoSCoW classification: Must Should Could WontHave
req:status	Requirement	xsd:string	Lifecycle state: Draft Validated Active Deprecated
req:conflictsWith	Requirement	Requirement	Requirements that cannot both be satisfied simultaneously
req:derivedFromUCA	UCAConstraint	UCARecord	NEW (STPA): Links a UCAConstraint back to the UCA analysis record that generated it
req:ucaType	UCAConstraint	xsd:string	NEW (STPA): The UCA failure mode: Missing Hazardous MisTimed WrongDuration
req:reversibilityClass	ControlRequirement	xsd:string	NEW (STPA): Reversibility classification: Reversible Conditionally-Reversible Irreversible
req:handshakeRequired	ControlRequirement	xsd:boolean	NEW (STPA): Whether this control requires human-in-the-loop approval before execution
req:timeout	NonFunctionalRequirement	xsd:duration	NEW (STPA): Maximum allowed duration before timeout and safe-state fallback
req:safeStateAction	ControlRequirement	xsd:string	NEW (STPA): The fallback action to take when a timeout or sequence violation is detected

3. Jobs-to-Be-Done as Semantic Anchors

The Jobs-to-Be-Done framework provides the motivational layer beneath requirements. A Job is not a task — it is the progress a persona is trying to make in a given context. Jobs are stable over time even when the features that serve them change. By anchoring requirements to Jobs, the architecture gains

an interpretive layer that the Semantic Gateway uses to validate whether a prompt is aligned with what the persona actually needs to accomplish.

In STAMP terms, a Job defines the operational goal that the top-level controller is trying to achieve. DesiredOutcomes define the success criteria against which all controlled processes are ultimately evaluated. JobBarriers define anticipated process model flaws — points where the controlled process may have incorrect beliefs about system state that, if uncorrected, will lead to unsafe control actions.

3.1 The Job Ontology Module

Class / Property	Type	Description
Job	Class	A functional unit of progress a persona hires a system to make
FunctionalJob	Subclass of Job	The core task objective — what the persona is literally trying to accomplish
EmotionalJob	Subclass of Job	How the persona wants to feel while doing it
SocialJob	Subclass of Job	How the persona wants to be perceived by others
DesiredOutcome	Class	A metric the persona uses to measure success — direction + metric + object + context
JobContext	Class	Situational circumstances that activate or shape the job
JobBarrier	Class	A friction point or obstacle preventing the persona from completing the job
UCARecord	Class	NEW (STPA): A formal record of one Unsafe Control Action, documenting the control action, failure mode type, context, and linked hazard/loss
job:performedBy	Property	Links a Job to the Persona class(es) that perform it
job:measuredBy	Property	Links a Job to its DesiredOutcome instances
job:hasUCA	Property	NEW (STPA): Links a Job to UCARecord instances defining what constitutes an unsafe control action for this job
job:safeState	Property	NEW (STPA): Links a Job to a SafeStateDefinition — the system state the gateway must return to when a UCA is detected

3.2 Illustrative JTBD Instantiation: Risk Manager Persona (STPA-Enhanced)

The following shows how a single persona's job hierarchy is represented as ontology individuals, now including UCA definitions and safe state specifications required by STPA:

JTBD Instance Graph — Risk Manager (STPA-Enhanced)

Persona: RiskManager_Senior

- FunctionalJob: 'Assess residual risk across active digital twin projects'
- JobContext: 'Quarterly board reporting cycle, limited time, stakeholder visibility HIGH'
- DesiredOutcome[1]: 'Minimize time to produce complete risk assessment from 3 days to 4 hours'
- DesiredOutcome[2]: 'Increase confidence that all ISO 31000 criteria are covered from 70% to 100%'

- JobBarrier[1]: 'Risk data scattered across 6 disparate systems'
- JobBarrier[2]: 'Inconsistent risk terminology between project teams and corporate risk function'
- UCARecord[1]: Control Action = 'Deliver risk assessment output'
UCA Type = Missing | Context = 'Agent enters retry loop' | Hazard = H-RDSG-003
UCAConstraint: 'If assessment output not delivered within 4 hours of job activation, trigger safe-state return'
- UCARecord[2]: Control Action = 'Deliver risk assessment output'
UCA Type = Hazardous | Context = 'Output contains unresolved terminology inconsistency'
UCAConstraint: 'All entity references must resolve to corporate risk ontology before delivery'
- UCARecord[3]: Control Action = 'Deliver risk assessment output'
UCA Type = MisTimed (Too Early) | Context = 'ISO 31000 validation not yet complete'
UCAConstraint: 'Output must not be delivered until sh:ISO31000Shape validation passes'
- UCARecord[4]: Control Action = 'Trigger legacy risk data export (Zone 6)'
UCA Type = Hazardous (Irreversible) | reversibilityClass = Irreversible | handshakeRequired = true
UCAConstraint: 'Simplex Action Gateway must confirm human approval before Zone 6 export executes'
- SafeStateDefinition: 'Return partial assessment with explicit completion status; notify steward; do not deliver incomplete output'
- JobRequirement: 'System must resolve risk entity terminology to corporate risk ontology before assessment'
- ControlRequirement: 'Assessment output must be validated against ISO_31000_Shape before delivery'

4. Requirements Capture Pipeline

Requirements enter the architecture through a structured elicitation and formalization pipeline. The pipeline transforms qualitative stakeholder input into formally typed ontology individuals, validated against the Requirements Ontology, before they are stored in the triplestore and indexed into the domain vector databases. The STPA-hardened edition adds explicit UCA elicitation at Stage 1, creating a formal bridge from hazard analysis to requirement generation.

4.1 Five-Stage Capture Pipeline (STPA-Enhanced)

Stage 1: Elicitation Interface (STPA-Enhanced)

A structured requirements capture form that prompts stakeholders using JTBD-derived question templates: 'Who are you?' (Persona), 'What are you trying to accomplish?' (Functional Job), 'What would success look like?' (Desired Outcome), 'What gets in your way?' (Job Barrier), 'What constraints must be respected?' (Compliance/Control). Outputs a draft Requirement Record.

NEW — STPA UCA Elicitation Block: For every Functional Job, stakeholders are additionally prompted to define the four STPA Unsafe Control Action types for the system's primary control action (the AI response for this job):

- Missing (Not Providing): 'In what context would it be harmful if the system did NOT produce a response for this job?' (e.g., agent timeout, infinite retry loop, silent failure)
- Hazardous (Providing): 'In what context would producing a response for this job be harmful or wrong?' (e.g., output contains unresolved terminology, response is based on stale data, compliance citation is absent)
- Mis-Timed (Too Early / Too Late / Out of Order): 'In what context would a response delivered at the wrong time be harmful?' (e.g., response delivered before validation completes, downstream API called before preconditions are met)
- Wrong Duration (Stopped Too Soon / Applied Too Long): 'In what context would a response that ends prematurely or continues processing past its intended scope be harmful?' (e.g., partial assessment delivered as complete, agent continues processing after cancellation)

UCA definitions are formalized as UCARecord individuals linked to the FunctionalJob. Each UCA automatically generates a UCAConstraint individual, which becomes a ControlRequirement with an enforcedBy linkage to the relevant SHACL shape or SWRL rule.

Stage 2: Ontology Classification

The draft Requirement Record is passed to the Semantic Gateway for classification. The gateway maps the stated need to Requirement subclasses, resolves persona and job references, and classifies any UCARecords into UCAConstraint individuals with ucaType, reversibilityClass, handshakeRequired, timeout, and safeStateAction properties populated. Outputs a typed RequirementDraft individual.

Stage 3: SHACL Validation of the Requirement Itself

The RequirementDraft is validated against sh:RequirementCompleteness shape. For jobs that produced UCARecords, the additional sh:UCACompleteness shape enforces: at least one UCARecord for each of the four UCA types, at least one UCAConstraint with enforcedBy linkage, safeStateAction populated for all Must-priority UCAs. Validation failures are returned to the elicitation interface with specific field guidance.

Stage 4: Stakeholder Review & Activation

Draft requirements enter a review workflow. Domain stewards validate domain assignment. STPA analysts review UCARecords for completeness and plausibility of hazard linkages. Compliance officers validate ComplianceRequirement instances. UCAConstraints derived from Irreversible UCAs trigger mandatory Simplex Action Gateway configuration review. Approved requirements transition to status=Active.

Stage 5: Vector DB Indexing

Active requirements — including UCAConstraints and SafeStateDefinitions — are embedded and indexed into the domain vector database(s) matching their inDomain property. UCA-derived embeddings are tagged with ucaType metadata, enabling the Semantic Gateway to retrieve relevant safe-state guidance when a UCA pattern is detected at runtime.

5. SHACL Shapes for Requirements

SHACL shapes governing requirements are stored as records in the relational database. This enables requirements governance teams to manage constraint evolution using familiar tooling without direct triplestore access. The STPA-hardened edition adds UCA completeness and reversibility enforcement shapes.

5.1 Core Requirement Shapes

Shape Name	Target Class	Enforced Constraints
sh:RequirementCompleteness	Requirement	req:forPersona exactly 1 req:inDomain min 1 req:priority in {Must, Should, Could, WontHave} req:status in {Draft, Validated, Active, Deprecated} req:hasCriterion min 1 req:version matches semantic version req:rationale min 1 (>20 chars)
sh:JobRequirementLinkage	JobRequirement	req:describesJob exactly 1 req:targetsOutcome min 1 job:measuredBy min 1 (DesiredOutcome with direction + metric) job:activatedIn min 1 (JobContext)
sh:UCACompleteness (NEW)	FunctionalJob	job:hasUCA min 4 — one per UCA type (Missing, Hazardous, MisTimed, WrongDuration) Each UCAREcord has: ucaType, context description, hazard linkage Each Must-priority UCAREcord has linked UCAConstraint with enforcedBy Each UCAConstraint has safeStateAction populated
sh:UCAConstraintShape (NEW)	UCAConstraint	req:derivedFromUCA exactly 1 req:ucaType in {Missing, Hazardous, MisTimed, WrongDuration} req:reversibilityClass in {Reversible, Conditionally-Reversible, Irreversible} req:enforcedBy min 1 req:safeStateAction min 1 req:handshakeRequired boolean populated
sh:IrreversibilityGate (NEW)	ControlRequirement	If req:reversibilityClass = Irreversible: req:handshakeRequired = true Simplex Action Gateway configuration reference required Human role authorized to approve must be specified Audit log destination must be specified
sh:TemporalRequirementShape (NEW)	NonFunctionalRequirement	If req:ucaType = Missing OR MisTimed: req:timeout must be populated (xsd:duration) req:safeStateAction must be populated Timeout value must be < overall job SLA from NFR performance threshold
sh:ComplianceRequirementShape	ComplianceRequirement	req:governedBy min 1 req:inDomain min 1 Applicable jurisdiction(s) linked req:enforcedBy min 1 when status=Active
sh:ControlRequirementShape	ControlRequirement	req:enforcedBy min 1 (SHACLShape OR SWRLRule) Testability evidence (SPARQL test query or test fixture) Owner (responsible data steward or engineer)
sh:ScenarioRequirementShape	ScenarioRequirement	req:scenarioContext exactly 1 Precondition assertions min 1 Trigger event or condition min 1 Expected behavior description min 1 At least 1 AcceptanceCriterion with SPARQL-evaluable test
sh:NonFunctionalRequirementShape	NonFunctionalRequirement	nfr:qualityDimension in {Performance, Reliability, Security, Usability, Maintainability, Scalability} nfr:measurableThreshold min 1 (numeric with unit) nfr:measurementMethod min 1 nfr:monitoringFrequency min 1

6. SWRL Rules Governing Requirement-Driven Behavior

SWRL rules stored in the relational DB fire on requirement class membership and job context to modify gateway behavior, activate domain agents, enforce acceptance criteria, and escalate conflicts. The STPA-hardened edition adds two new rule families: the TMP (Temporal) series governing time-based behaviors and safe-state fallback, and the UCA series directly encoding UCA constraints as behavioral governors.

6.1 Core Requirement Rules (REQ Series — Enhanced)

Rule ID	Trigger Condition	Inferred Consequence	Architectural Effect
REQ-001	SPO persona_id matches Persona P AND P performs FunctionalJob J AND J has JobContext C active	Attach J's DesiredOutcome metrics as success criteria to CAD	Gateway injects outcome measurement instructions into AI Directive
REQ-002	SPO intent resolves to domain D AND D has Active ScenarioRequirement S with matching preconditions	S's AcceptanceCriteria become validation gates on agent response	Service Facade validates agent output against S before delivery
REQ-003	Request targets Entity E AND E is in scope of ComplianceRequirement CR AND CR has ControlRequirement CC	CC's enforcing SHACLShape is added to active constraint set	Gateway retrieves and applies CC's shape during compilation
REQ-004	FunctionalRequirement FR has status=Active AND FR has no enforcedBy linkage AND FR priority=Must	Flag FR as GOVERNANCE_GAP and attach to CAD as warning	Response includes governance gap notification for steward review
REQ-005	Persona has JobBarrier B with barrier_type=TerminologyInconsistency AND B applies to domain D	Force ontology disambiguation on all D-domain terms in SPO	Gateway triggers disambiguation before SHACL evaluation
REQ-006	ScenarioRequirement S has status=Active AND SPO context matches S's scenarioContext	Retrieve S's scenario-specific vector DB entries as priority context	Agent receives scenario-specific results ranked above general results
REQ-007	NonFunctionalRequirement NFR applies to intent AND quality_dimension=Performance AND threshold=T	Attach performance SLA T to CAD execution parameters	Agent invocation is governed by T as timeout/resource ceiling
REQ-008	Two Active Requirements R1 and R2 have conflictsWith each other AND both triggered by same SPO	Halt compilation, return REQUIREMENT_CONFLICT with citations	Human resolution required — conflict logged to governance queue

6.2 Temporal Safety Rules (TMP Series — NEW)

The TMP rule series implements STPA's time-based UCA coverage. Every rule in this series addresses either the 'Too Early / Too Late / Out of Order' UCA type or the 'Stopped Too Soon / Applied Too Long' UCA type. These rules transform passive timeout thresholds into active behavioral governors with explicit safe-state fallback actions.

Rule ID	Trigger Condition	Inferred Consequence	Architectural Effect
TMP-001	Agent A has begun execution of Step S in FunctionalJob J AND S has timeout T defined in UCAConstraint AND elapsed time > T	Halt Agent A execution; activate req:safeStateAction defined in the UCAConstraint linked to J	Gateway terminates agent invocation; Service Facade returns safe-state message to user; RequirementsFinding logged
TMP-002	FunctionalJob J has Step S1 that must precede Step S2 AND S2 is initiated before S1 status = Complete	Return SEQUENCE_VIOLATION for S2; attach to CAD as blocking error; halt pipeline at S2	Service Facade cannot deliver S2 output until S1 completes; violation logged as UCA-MisTimed finding
TMP-003	Zone 6 downstream API call C is annotated with ControlRequirement CR AND CR requires predecessor P (prior validation step) AND P has not fired	Halt Zone 6 call C; return PRECONDITION_VIOLATION; route to Service Facade for human resolution	Simplex Action Gateway blocks call until P is confirmed; prevents out-of-sequence external write
TMP-004	Agent A has completed primary job J AND has not received explicit cancellation AND continues processing beyond scope boundary defined in UCAConstraint (WrongDuration)	Inject SCOPE_EXCEEDED signal into CAD; trigger agent termination after delivering scope-boundary response	Prevents 'applied too long' UCA — agent scope is formally bounded by requirement, not just prompt length
TMP-005	SPO for FunctionalJob J arrives AND J has a predecessor Job P with status != Complete AND req:dependsOn linkage exists	Halt J compilation; return DEPENDENCY_UNSATISFIED with P citation and completion status	Gateway enforces job sequencing — partial job states cannot trigger downstream AI actions
TMP-006	Agent A is in retry loop (attempt count > configured max) AND no successful SPO_COMPLETE signal received	Fire TMP-001 safe-state fallback; escalate to RequirementsReview; notify governance steward	Prevents infinite retry pattern (Missing UCA) — explicit retry ceiling with governance escalation

6.3 UCA Enforcement Rules (UCA Series — NEW)

The UCA rule series directly encodes identified Unsafe Control Action constraints as executable gateway behaviors. Every UCAREcord in the triplestore generates a corresponding UCA rule that the gateway fires when runtime conditions match the UCA's defined context.

Rule ID	Trigger Condition	Inferred Consequence	Architectural Effect
UCA-001	SPO matches FunctionalJob J AND J has UCAREcord with ucaType=Missing AND runtime context matches UCA context description AND no agent response token received within req:timeout	Activate req:safeStateAction for this UCAConstraint; log UCA-Missing event as RequirementsFinding	Gateway treats timeout as a formally identified hazard, not a technical error — safe state is policy-defined, not default
UCA-002	Agent response R is generated AND R contains entity references that do not resolve in domain ontology AND active UCAREcord for J has ucaType=Hazardous for this condition	Reject R; return to agent with ONTOLOGY_RESOLUTION_FAILURE and structured correction instructions	Service Facade enforces hazardous-output constraint — unresolved terminology is a formally identified hazard

UCA-003	AcceptanceCriteria for ScenarioRequirement S have not all passed AND Service Facade receives delivery request for response R AND UCAREcord for J has ucaType=MisTimed (TooEarly)	Block R delivery; annotate R with pending criteria and required completion steps; return to agent	Prevents premature delivery — 'too early' UCA is formally gated by criterion completion, not agent readiness
UCA-004	Agent has produced partial response Rp AND job scope has been exceeded (WrongDuration UCA context) AND Rp does not carry explicit partial-completion markers	Append explicit PARTIAL_RESPONSE marker and completion-status payload to Rp; deliver with scope-exceeded notice	Prevents 'stopped too soon' UCA without blocking delivery — scope boundaries are made explicit to downstream consumers
UCA-005	Zone 6 action C has reversibilityClass=Irreversible AND human handshake token not present in CAD AND action is about to execute	Block C execution; return HANDSHAKE_REQUIRE D to Service Facade; suspend Zone 6 call pending approval	Simplex Action Gateway enforces irreversibility constraint — no human-approved token, no irreversible action

6A. Simplex Action Gateway: Zone 6 Irreversibility Enforcement

Prof. Leveson's Simplex Architecture principle states that once an irreversible action is executed, post-action feedback is useless. In enterprise AI contexts, the equivalent of launching a missile is triggering a financial trade, deleting master data, publishing a regulatory submission, or initiating an external contractual commitment. The Simplex Action Gateway implements this principle as a formal architectural boundary at Zone 6.

6A.1 Reversibility Classification

Every Zone 6 action — downstream API call, write operation, or integration trigger — must be classified using the req:reversibilityClass property on its governing ControlRequirement. The Simplex Action Gateway enforces different policies for each class:

Reversibility Class	Definition	Gateway Policy	Example
Reversible	Action can be fully undone without loss — idempotent reads, soft-delete with restore, draft state writes	Proceed on CAD approval. Log action with requirement ID.	Query a read-only data API. Create a draft record in a workflow system.
Conditionally-Reversible	Action can be undone within a defined time window or with bounded consequences	Proceed on CAD approval AND automated consequence warning. Human notification (non-blocking) sent. Undo window initiated.	Submit a draft report for internal review. Create a pending financial entry (not yet posted).
Irreversible	Action cannot be undone or the cost of undoing exceeds acceptability threshold	BLOCK execution. Return HANDSHAKE_REQUIRE D to Service Facade. Suspend Zone 6 call. Require explicit human approval token from authorized role before proceeding.	Post a regulatory filing. Execute a financial transaction. Delete a master data record. Publish an external API response to a production system.

6A.2 Handshake Protocol for Irreversible Actions

When UCA-005 fires or the Simplex Action Gateway classifies a Zone 6 call as Irreversible, the following protocol executes:

- **Step 1 — Block:** The gateway suspends execution of the Zone 6 call and generates a HANDSHAKE_REQUIRED event with: the action description, the ControlRequirement ID governing it, the reversibility classification, and the estimated consequence if executed.
- **Step 2 — Route:** The Service Facade routes the HANDSHAKE_REQUIRED event to the human role specified in the ControlRequirement's owner field. This role must be pre-authorized in the Identity Ontology.
- **Step 3 — Confirm:** The authorized human reviews the action and returns an explicit approval token containing: approver identity, timestamp, confirmation of understanding of consequences, and any override rationale.
- **Step 4 — Execute:** The Simplex Action Gateway validates the approval token (checks identity, authorization, and freshness — tokens expire after configured TTL). If valid, the Zone 6 call executes with the approval token attached to the audit record.
- **Step 5 — Log:** All Irreversible action executions are appended to an immutable audit log with: approver identity, approval token hash, action payload hash, Zone 6 endpoint, requirement ID chain, and timestamp.

If the approval token is absent, expired, or issued by an unauthorized role, the Simplex Action Gateway returns APPROVAL_INVALID and the action does not execute under any circumstances.

7. Gateway Integration: How Requirements Shape Prompt Compilation

When the Semantic Gateway receives a Structured Prompt Object, requirement resolution becomes Stage 0 — before ontology resolution, before SHACL evaluation, before PEP orchestration. Requirements define the lens through which everything else is interpreted. The STPA-hardened edition adds UCA checking and temporal constraint activation to this pipeline.

7.1 Updated Gateway Compilation Pipeline

Stage 0 — Requirement Resolution (Enhanced)

The gateway queries the triplestore for all Active requirements matching: (a) the authenticated persona, (b) the resolved intent_tag mapped to a FunctionalJob class, and (c) the entity domain. The result is a Requirement Context Bundle (RCB). NEW: the RCB now includes the full UCA profile for the matched FunctionalJob — all four UCA types with their contexts, timeouts, safe-state actions, and reversibility classifications. The UCA profile is the gateway's process model for what constitutes unsafe behavior in this interaction.

Stage 1 — Ontology Resolution (Enhanced)

Entity resolution now includes job context injection: resolved entities are annotated with the persona's active JobContext, which modifies property visibility, priority weighting, and barrier context. NEW: any entity that is flagged in a JobBarrier with barrier_type=TerminologyInconsistency is immediately elevated to UCA-002 monitoring status — the gateway treats unresolved terminology as a live hazardous output risk for this interaction.

Stage 2 — SHACL Evaluation (Enhanced)

SHACL evaluation now includes UCA-derived shapes: UCAConstraint individuals with enforcedBy linkage inject their shapes into the active evaluation set. Each shape result is annotated with the UCARecord that activated it — creating a traceable chain from enforcement event back to the original hazard analysis.

Stage 3 — SWRL Rule Execution (Enhanced)

REQ-001 through REQ-008 fire as before. NEW: TMP-001 through TMP-006 fire to activate temporal constraints and enforce job sequencing. UCA-001 through UCA-005 fire to detect and contain unsafe control action patterns before they reach the delivery layer.

Stage 4 — PEP Orchestration (Enhanced)

PEP evaluation is now requirement-aware and UCA-aware. Each PEP receives the RCB including the UCA profile and uses it to contextualize its verdict. NEW: Any PEP receiving a Zone 6 dispatch request must check the reversibilityClass of the governing ControlRequirement before approving. Irreversible actions cannot receive a PEP ALLOW verdict — they require explicit Simplex Action Gateway handshake.

Stage 5 — CAD Compilation (Enhanced)

The final Compiled AI Directive now includes: the Requirement Context Bundle (requirements governing this interaction), DesiredOutcome metrics as explicit success criteria, AcceptanceCriteria as response validation gates, scenario-specific retrieval priority instructions, governance gap flags, the UCA monitoring profile (active UCAs with their context patterns and safe-state actions), temporal constraint declarations (TMP rule activations with timeout values), and for any Zone 6 calls: reversibilityClass and handshake-required flags.

7A. Human-System Process Model Synchronization

STAMP explicitly recognizes that the human controller's process model can be flawed. If the stakeholder misunderstands the system's capabilities, selects the wrong JTBD, or defines acceptance criteria that do not reflect actual risk boundaries, the entire downstream architecture will perfectly execute the wrong control structure. Human process model divergence is not an AI failure — it is a system hazard that must be detected and treated as such.

7A.1 Human-System Misalignment as a STAMP Hazard

The following system hazard is formally added to the RDSG STPA analysis scope:

System-Level Hazard: H-RDSG-HSM-001

H-RDSG-HSM-001: The Semantic Gateway executes a correct control structure against a flawed human process model, producing outputs that satisfy all formal requirements but fail to address the stakeholder's actual need — and the divergence is not detected or flagged.

Loss association: L-RDSG-002 (Loss of stakeholder trust in AI governance), L-RDSG-004 (Compliance gap created by requirements that do not correctly reflect regulatory intent), L-RDSG-005 (Operational harm from systematically-correct-but-contextually-wrong AI outputs).

System-Level Constraint SC-HSM-001: The architecture must detect and flag cases where the human controller's process model (as expressed through elicited requirements and Structured Prompt Objects) is inconsistent with the system's active control structure, and must trigger a structured resolution before continuing to execute.

7A.2 Detection Mechanisms

The following patterns trigger a HUMAN_SYSTEM_MISALIGNMENT event in the governance queue:

Misalignment Signal	Detection Point	Gateway Response
Repeated SPO conflicts with active ScenarioRequirement: the persona's prompts consistently request behaviors outside the active scenario's defined scope	Stage 0 Requirement Resolution — RCB conflict detection	Flag HUMAN_SYSTEM_MISALIGNMENT; annotate CAD with misalignment notice; prompt persona with scoped clarification question; escalate to steward if pattern persists across 3+ interactions
JTBD mismatch: the resolved FunctionalJob does not match the intent_tag inferred from the SPO semantic embedding (cosine distance exceeds threshold)	Stage 0 RCB resolution — job matching step	Present matched vs. inferred job to persona for confirmation; do not proceed until persona confirms job alignment or selects alternative job
AcceptanceCriteria systematic failure: more than N failures of a Must-priority criterion within the review window defined in Section 11.2 — suggesting the criterion may not reflect true operational conditions	Feedback Loop (Section 11.2) — RequirementsFindings aggregation	Trigger RequirementsReview; route to steward for criterion calibration OR capability improvement decision; tag all CADs with CRITERIA_UNDER_REVIEW until resolved
Persona requests a capability the system explicitly does not provide (no FunctionalRequirement with status=Active covers the requested behavior)	Stage 0 RCB resolution — no matching FunctionalRequirement found	Return CAPABILITY_BOUNDARY notice with gap description; log as OntologyGap candidate; do not attempt approximate fulfillment without explicit human authorization
Elicitation UI receives JTBD responses that are internally inconsistent (DesiredOutcome contradicts JobBarrier, or scenario context contradicts defined constraints)	Stage 3 SHACL Validation — sh:JobRequirementLinkage shape	Return validation failure to Stage 1 with specific conflict description; prompt stakeholder to resolve contradiction before requirement is classified

7A.3 Process Model Synchronization Protocol

When a HUMAN_SYSTEM_MISALIGNMENT event is escalated to the governance queue, the following protocol applies:

- Step 1 — Characterize: The gateway logs the specific divergence pattern: which requirement was violated, how many times, under which persona/job combination, and what the SPO patterns suggest about the stakeholder's actual mental model of the system.
- Step 2 — Triage: The domain steward reviews the characterization and determines the root cause: (a) the stakeholder's mental model needs updating (training/documentation gap), (b) the requirement needs updating (specification gap), or (c) the AI capability needs updating (development gap).
- Step 3 — Resolve: The appropriate corrective action is taken in the corresponding architectural zone. Root cause (a) triggers elicitation UI improvement or stakeholder training. Root cause (b) triggers a RequirementsReview. Root cause (c) triggers a capability development backlog item.
- Step 4 — Verify: After the corrective action, the misalignment pattern is monitored for recurrence. If the same divergence pattern returns within the next 30-day window, the triage determination is re-examined — the root cause may have been misidentified.

8. Requirements-Driven Vector Database Indexing

Domain vector databases are not generic semantic search indices. With requirements integration, they become structured knowledge stores organized by persona, job, and scenario. The STPA-hardened edition adds UCA-pattern tagging and safe-state guidance indexing to the vector DB architecture, enabling the gateway to retrieve UCA-relevant guidance at runtime without additional triplestore roundtrips.

8.1 Extended Indexing Taxonomy (STPA Additions)

Metadata Field	Source	Retrieval Role
persona_classes	req:forPersona	Filter: only retrieve content relevant to the querying persona's role
job_ids	req:describesJob	Boost: content indexed for the active job retrieved at higher ranking
desired_outcome_ids	req:targetsOutcome	Boost: content directly relevant to active desired outcomes is priority-ranked
domain_id	req:inDomain	Hard filter: content outside the active domain is excluded unless cross-domain flag is set
scenario_context_ids	req:scenarioContext	Exact match boost: scenario-specific content is ranked highest when context matches
compliance_refs	req:governedBy	Compliance content tagged with active obligations is included regardless of similarity score
uca_type_tags (NEW)	UCARecord.ucaType	Runtime UCA monitoring: chunks containing safe-state guidance for the active UCA types are retrieved as priority context when a UCA pattern is detected
safe_state_ids (NEW)	SafeStateDefinition URIs	Safe-state execution: when a TMP or UCA rule fires, the matching safe-state guidance chunk is retrieved

		unconditionally and prepended to the agent's correction context
reversibility_class (NEW)	req:reversibilityClass	Zone 6 gating: Zone 6 call records tagged Irreversible are excluded from automated execution paths; they are routed to the Simplex Action Gateway retrieval context instead
acceptance_criteria_ids	req:hasCriterion	Validation: retrieved content cross-referenced against acceptance criteria at agent evaluation time
nfr_quality_tags	NonFunctionalRequirement qualityDimension	Performance filtering: retrieval latency targets derived from active NFR performance thresholds
version_effective_from	req:version + effective date	Temporal filter: content indexed under deprecated requirements is excluded from active queries

9. Requirements Coverage Across Domains

The following table illustrates how the STPA-hardened requirements framework applies across key enterprise domains. Each domain now includes defined UCAs for its primary AI control actions, with Simplex Action Gateway configuration for irreversible operations.

Domain	Primary Irreversible Zone 6 Actions	Key UCA Constraints (STPA)
Risk Management	Board-level risk report publication; regulatory capital threshold override	Missing: agent timeout before 4-hour SLA → safe-state partial report Hazardous: output with unresolved terminology → ONTOLOGY_RESOLUTION_FAILURE MisTimed (early): output before ISO 31000 validation → block delivery WrongDuration: assessment scope exceeded → SCOPE_EXCEEDED marker
Data Governance	GDPR Art. 30 processing record submission; cross-border transfer execution; master data deletion	Missing: certification job abandonment → notify steward Hazardous: output asserting certified status for non-validated data product → COMPLIANCE_CITATION_REQUIRED MisTimed: cross-border transfer before jurisdiction-specific consent confirmed → block Zone 6 call
Security & Access	Privileged access grant execution; firewall rule push; credential revocation	Irreversible: privileged access grant requires dual-control handshake (two authorized roles) MisTimed: threat correlation output before P1 SLA (30s) exceeded → TMP-001 safe-state escalation Missing: zero-day advisory not delivered within defined window → TMP-006 retry ceiling
Compliance & Legal	External regulatory filing submission; signed contract execution	Irreversible: all external regulatory submissions require legal review gate (ProcessRequirement) + Simplex handshake MisTimed (out of order): submission before legal review step complete → TMP-002 SEQUENCE_VIOLATION Hazardous: submission without SOX citation → UNGROUNDED flag
AI Governance	Model deployment to production; bias assessment approval; model retirement	Irreversible: production model deployment requires human-in-the-loop checkpoint MisTimed: revalidation after model drift detected must complete within 90-day window (TMP-001) WrongDuration:

		explainability report generation bounded by 5-minute NFR (TMP-004)
Analytics & Reporting	SOX regulatory disclosure; financial restatement publication	Irreversible: financial disclosures require CFO-level handshake token Hazardous: dashboard output without statistical validity certification → ANALYTIC_QUALITY_FAILURE MisTimed: restatement re-run must complete with audit trail before prior-period figures are published

10. Acceptance Criteria as Automated Validation Gates

AcceptanceCriterion individuals are executable tests evaluated by the Service Facade before any response is delivered. The STPA-hardened edition adds UCA-specific criterion types that directly test for unsafe control action patterns — not just structural completeness of the output.

10.1 Acceptance Criterion Types (Enhanced)

Criterion Type	Evaluation Method	Example
Structural Completeness	JSON Schema or SHACL shape on response JSON	Risk assessment must contain: risk_id, likelihood_score, impact_score, residual_risk, control_recommendations, ISO_31000_clause_reference
Ontology Consistency	SPARQL ASK on response-derived triples	All entity references must resolve to valid individuals in domain ontology — no undefined terms (UCA-002 enforcement)
Outcome Metric Coverage	Keyword/entity presence check mapped to DesiredOutcome metrics	Response must address all DesiredOutcome metrics in the RCB — missing metric triggers specific clarification request
Compliance Citation	Reference validation against ComplianceRequirement standards	Compliance analysis must cite applicable standard sections — ungrounded assertions flagged
UCA-Missing Test (NEW)	Timeout check: response token received within req:timeout fallback: UCA-001 safe state	Assessment job: agent response must arrive within configured timeout; absence triggers TMP-001 and safe-state delivery
UCA-MisTimed Test (NEW)	Prerequisite completion check: all predecessor criteria must pass before this criterion evaluates block delivery if any fail	Risk output: ISO 31000 validation shape must have returned PASS before delivery attempt — TMP-002 SEQUENCE_VIOLATION otherwise
UCA-WrongDuration Test (NEW)	Scope boundary check: response entity scope matches job scope definition; PARTIAL_RESPONSE marker present if scope boundary was reached before completion	Assessment scope: response entities must be within defined portfolio scope; if scope boundary reached, marker must be explicit
Irreversibility Gate (NEW)	Zone 6 pre-check: reversibilityClass evaluated before action dispatch; handshake token validated if Irreversible	Regulatory filing dispatch: valid human approval token must be present and unexpired before Zone 6 filing endpoint is called
NFR Quality Gate	Latency measurement, response length, readability score	Response delivered within performance NFR threshold; length proportional to complexity NFR specification

Scenario-Specific Test	Custom SPARQL or Python evaluation against scenario fixture	Board reporting: format matches board template schema, confidence intervals included, no raw data exposed
------------------------	---	---

11. Requirements Governance Operations

11.1 Requirements Lifecycle Management

- Versioning: Requirements follow semantic versioning aligned to ontology releases. Breaking changes to UCAConstraints or SafeStateDefinitions require a major version increment and full STPA re-review of affected jobs.
- UCA Review Cadence: UCARecords for Must-priority jobs are reviewed quarterly or whenever a new AI capability is deployed that extends the job's scope. Review produces updated UCAConstraints and may generate new TMP or UCA rules.
- Deprecation: Deprecated requirements — including UCAConstraints — are retained in the triplestore with status=Deprecated. Active UCA monitoring rules that reference a deprecated UCARecord are automatically flagged for replacement before the deprecation is finalized.
- Simplex Gateway Audit: All Irreversible action executions are auditable by compliance officers through the immutable Zone 6 audit log. Required fields: approver, timestamp, action payload hash, approval token hash, governing ControlRequirement ID chain.
- Coverage Dashboard (Enhanced): Adds STPA coverage metrics: % of active FunctionalJobs with all four UCA types defined, % of Must-priority UCAs with enforcedBy TMP or UCA rule, % of Irreversible Zone 6 actions with Simplex Gateway configuration, % of ScenarioRequirements with SafeStateDefinition populated.

11.2 Feedback Loop: Requirements Evolution

The architecture supports closed-loop requirements evolution. When an agent response fails an AcceptanceCriterion, the failure is logged as a RequirementsFinding linked to the failing criterion. NEW: UCA-type criterion failures are additionally classified by UCA type (Missing, Hazardous, MisTimed, WrongDuration) and trigger UCA-pattern aggregation analysis. If a UCA pattern recurs with sufficient frequency, a UCA_PATTERN_REVIEW is triggered — prompting STPA analysts to determine whether the UCA's causal scenario has changed, whether the constraint is correctly specified, or whether a new UCA type is emerging.

- Finding → Review Threshold: Must-priority criteria trigger review after 3 failures within 30 days; Should-priority after 10 failures. UCA-type failures trigger immediate steward notification regardless of priority.
- Human-System Misalignment Feedback: Patterns from Section 7A feed into this loop. HUMAN_SYSTEM_MISALIGNMENT events are aggregated weekly and surfaced to the elicitation team as UCA-causal candidates — misalignment patterns may indicate that the original UCA analysis did not adequately model human process model failure modes.
- Temporal Rule Calibration: TMP rule timeout values and sequence constraints are calibrated against operational performance data. If TMP-001 fires more than a configured threshold of times in a period, the timeout value in the governing NonFunctionalRequirement is flagged for review — it may be too tight for current system performance, or system performance may have degraded.

12. Integration Summary: What Changes in Each Architectural Zone

Zone	What the STPA/STAMP Integration Adds
Zone 1: Prompt Engineering	SPO gains uca_profile_query field; elicitation UI adds STPA UCA elicitation block (4 UCA types per job); persona registry links to UCAREcord instances; prompt templates versioned against UCAConstraint versions; human process model synchronization triggers when JTBD mismatch detected
Zone 2: Semantic Gateway	Stage 0 requirement resolution includes full UCA profile in RCB; TMP and UCA SWRL rules fire in Stage 3; CAD gains: UCA monitoring profile, temporal constraint declarations, Zone 6 reversibility flags, human-system misalignment status; safe-state guidance chunks retrieved unconditionally on UCA rule fire
Zone 3: Ontology & Rules Store	UCAREcord and UCAConstraint classes added to Requirements Ontology; SafeStateDefinition class added; sh:UCACompleteness and sh:UCAConstraintShape and sh:IrreversibilityGate and sh:TemporalRequirementShape added; TMP-001 through TMP-006 rules added; UCA-001 through UCA-005 rules added; new properties: ucaType, reversibilityClass, handshakeRequired, timeout, safeStateAction
Zone 4: Service Facade	UCA-type AcceptanceCriteria evaluated on all responses; Irreversibility Gate criterion evaluated before any Zone 6 dispatch; HUMAN_SYSTEM_MISALIGNMENT detection logic added to context-content mapper; RequirementsFindings classified by UCA type for aggregation analysis
Zone 5: AI Agents + Vector DBs	Vector DB entries carry uca_type_tags and safe_state_ids metadata; safe-state guidance chunks retrieved unconditionally on TMP/UCA rule fire; reversibility_class metadata governs Zone 6 routing; RDQE gains UCA-pattern retrieval bias for active UCA monitoring contexts
Zone 6: Legacy Integration	Simplex Action Gateway implemented at Zone 6 boundary; reversibilityClass evaluated for every action before execution; Irreversible actions suspended pending human handshake; handshake protocol enforced (Step 1–5 in Section 6A.2); immutable audit log maintained for all Irreversible action executions; TMP-003 precondition enforcement prevents out-of-sequence Zone 6 calls

13. STPA Analysis Scope: Losses, Hazards, and System-Level Constraints

The following defines the formal STPA analysis scope for the RDSG as a governed AI system. This scope is a living document — it must be updated whenever new AI capabilities are added, new domains are onboarded, or new Zone 6 integrations are commissioned.

13.1 System Losses (RDSG Scope)

Loss ID	Loss Description	Stakeholder
L-RDSG-001	Harm to regulated persons or entities from AI-generated outputs that are incorrect, incomplete, or non-compliant with applicable regulations	Regulated entities, end users, oversight bodies
L-RDSG-002	Loss of stakeholder trust in the AI governance architecture, reducing adoption and organizational value of the platform	Business sponsors, governance teams, domain stewards
L-RDSG-003	Regulatory penalty, legal liability, or audit failure arising from a failure of the AI system to enforce a compliance requirement	Legal, compliance, and risk functions

L-RDSG-004	Execution of an irreversible enterprise action (financial transaction, regulatory filing, data deletion) without proper human authorization	Finance, legal, operations, executive sponsors
L-RDSG-005	Systematic delivery of contextually-wrong outputs that satisfy formal requirements but fail the stakeholder's actual need — undetected until operational harm occurs	Domain stewards, end users, oversight functions
L-RDSG-006	Compromise of data confidentiality, integrity, or access controls through AI-generated outputs that exceed persona authorization scope	Security, privacy, and data governance functions

13.2 System-Level Hazards and Constraints

Hazard ID	Hazard Description	System-Level Constraint	Losses
H-RDSG-001	Semantic Gateway delivers an AI response that violates an active ControlRequirement without detection	SC-001: All agent responses must be validated against all active AcceptanceCriteria in the RCB before delivery. Non-passing responses must not be delivered.	L-1, L-2, L-3
H-RDSG-002	Zone 6 executes an irreversible action without human authorization	SC-002: No Irreversible Zone 6 action may execute without a valid human approval token issued by an authorized role. Absence of token must block execution unconditionally.	L-4
H-RDSG-003	Agent enters a state (timeout, retry loop, scope overflow) that prevents delivery of a required response within the defined job SLA, without safe-state fallback	SC-003: All FunctionalJobs must have defined timeout thresholds and safe-state actions. TMP rules must fire before SLA expiry. Safe-state must be delivered to the user.	L-2, L-5
H-RDSG-004	AI response is delivered before all required predecessor steps (validation, disambiguation, precondition checks) have completed — creating a 'too early' output hazard	SC-004: AcceptanceCriteria must include prerequisite completion checks for all MisTimed UCA types. Service Facade must block delivery until all predecessor criteria pass.	L-1, L-3
H-RDSG-005	Human process model divergence causes systematically-correct-but-wrong outputs to be generated across repeated interactions without detection	SC-005: Gateway must detect and flag human-system misalignment patterns (Section 7A). HUMAN_SYSTEM_MISALIGNMENT events must be escalated to stewards within the defined SLA.	L-5
H-RDSG-006	AI response exceeds persona data authorization scope, exposing confidential data to unauthorized individuals	SC-006: Persona access scope from Identity Ontology must be enforced as a hard filter at vector DB retrieval (Zone 5) and validated by Persona Scope Compliance criterion at Service Facade.	L-6

14. Scaling with Emergent Complexity: STPA as a Living Practice

The RDSG was designed to absorb emergent complexity — the property by which AI-extended enterprise systems develop behaviors that were not present in or predictable from their individual components. STAMP provides the theoretical basis for this claim: in systems where components interact in unexpected ways, safety (and controllability) must be designed into the control structure rather than predicted from component reliability statistics.

As AI capabilities expand — from single-turn completions to multi-step agentic chains to fully autonomous AI systems managing complex workflows — the RDSG control structure scales through the following mechanisms:

14.1 Adding New AI Capabilities

- Each new AI capability (new agent type, new vector DB partition, new Zone 6 integration) is a new controlled process that inherits the existing constraint hierarchy. Onboarding requires: defining the FunctionalJobs it serves, completing STPA UCA elicitation for those jobs, specifying SafeStateDefinitions, and configuring Simplex Action Gateway reversibility classification for any Zone 6 actions it triggers.
- Emergent inter-agent behaviors — where two agents interact in ways not individually specified — must be modeled as new control loops in the STAMP control structure diagram and analyzed for UCAs at their interaction boundary.

14.2 Multi-Agent Agentic Chains

- In agentic chains, each agent-to-agent handoff is a control action in the STAMP hierarchy. The sending agent is a controller; the receiving agent is a controlled process. All four UCA types must be analyzed at each handoff boundary — including timing (what if Agent B does not receive Agent A's output within the expected window?) and scope (what if Agent A passes a larger context than Agent B is authorized to process?).
- REQ-008 (conflict detection) and TMP-002 (sequence enforcement) together provide the foundational governance for multi-agent chains. Dedicated multi-agent SWRL rules should be developed for each agentic chain pattern as it is deployed.

14.3 Defense and High-Assurance Contexts

- For defense applications (RAVEN-3, HERALD-1 and analogous programs), the RDSG STPA scope must extend to include mission-level losses (L-RDSG-004 equivalent: loss of mission, loss of human life, loss of asset) and must apply mil-spec reversibility classifications. The Simplex Action Gateway handshake protocol maps directly to existing dual-control and positive-control requirements in defense doctrine.
- The Composition Theorem and MTBH metric (developed in the companion morphism-grounded compositional assurance architecture) provide the formal mathematical foundation for quantifying assurance case confidence across composed RDSG control structures in multi-system-of-systems contexts.

Document Status and Version

Version: 2.0 DRAFT | Status: STPA/STAMP Integration Review

Supersedes: RDSG v1.0 (single-session draft, pre-STPA)

Next Steps: (1) STPA analyst review of UCA completeness in Section 13.2 | (2) Domain steward validation of SafeStateDefinitions for each enterprise domain | (3) Simplex Action Gateway reversibility classification review for all active Zone 6 integrations | (4) Integration with Morphism-Grounded Compositional Assurance Architecture (MTBH, Composition Theorem) | (5) Submission preparation for AI4SE & SE4AI Workshop and DARPA BAA contexts

Maintained by: The Value Enablement Group, LLC